Starter

For each of the following functions, write down the **Big-O notation**:

1. $a(x) = x + 10$
2. $b(t) = 4t^3 + 6t^2$
3. $c(t) = 2^\pi + 1$
4. $d(n) = 5 \log_2 n + 0.6n$
5. $e(n) = 2^n + n^2$
6. $f(n, t) = 5nt + \log_2 n$

Starter

For each of the following functions, write down the **Big-O notation**:

1. $a(x) = x + 10$
2. $b(t) = 4t^3 + 6t^2$
3. $c(t) = 2^\pi + 1$
4. $d(n) = 5\log_2 n + 0.6n$
5. $e(n) = 2^n + n^2$
6. $f(n, t) = 5nt + \log_2 n$

1. $O(x)$
2. $O(t^3)$
3. $O(1)$
4. $O(n)$
5. $O(2^n)$
6. $O(nt)$

Starter 2

For each of the following functions, write down the **Big-O notation**:

1. $a(x) = 4x + 1$
2. $b(t) = 10t^2 + t^4$
3. $c(t) = \log_2 t + t$
4. $d(n) = 7n^7 + 2^n$
5. $e(n) = 2n^2 + 3n + 4n \log_2 n$
6. $f(n, t) = 5nt + \log_2 n + t^4$

Starter 2

For each of the following functions,
write down the **Big-O notation**:

1. $a(x) = 4x + 1$
2. $b(t) = 10t^2 + t^4$
3. $c(t) = \log_2 t + t$
4. $d(n) = 7n^7 + 2^n$
5. $e(n) = 2n^2 + 3n + 4n \log_2 n$
6. $f(n, t) = 5nt + \log_2 n + t^4$

1. $O(x)$
2. $O(t^4)$
3. $O(t)$
4. $O(2^n)$
5. $O(n^2)$
6. $O(t^4)$

Common Algorithms Recap

Algorithm	Size of Problem(n)	Time Comp
Linear Search	(1)	$O(n)$
Binary Search	Length of search list	(2)
Binary Tree Search	(3)	$O(\log n)$
Bubble Sort	Length of list to sort	(4)
Merge Sort	Length of list to sort	(5)

Common Algorithms Recap

Algorithm	Size of Problem(n)	Time Comp
Linear Search	Length of search list	$O(n)$
Binary Search	Length of search list	$O(\log n)$
Binary Tree Search	Length of search list	$O(\log n)$
Bubble Sort	Length of list to sort	$O(n^2)$
Merge Sort	Length of list to sort	$O(n \log n)$

Units for measuring time

We use Big O notation when it's not sensible to know a unit of time. Too many factors are involved.

Units for measuring time

We use Big O notation when it's not sensible to know a unit of time. Too many factors are involved.

What can you think of?

Units for measuring time

We use Big O notation when it's not sensible to know a unit of time. Too many factors are involved.

Could count how many times each operation in an algorithm is run, but that'll take ages.

Order of Growth

It is sufficient to identify the operation contributing most to the total running time and compute the number of times this operation is carried out.

This is sometimes called the **basic operation**.

For searching and sorting algorithms, we often count the **number of comparisons** made. In this case, the basic operation is a comparison between values.

Order of Growth

Estimated running time for a **linear** algorithm can be expressed as:

$$\text{Time taken} \approx \text{Basic Op. Time} \times \text{Op. Count}$$

What would this look like for a **quadratic** algorithm?

Order of Growth

Estimated running time for a **linear** algorithm can be expressed as:

$$\text{Time taken} \approx \text{Basic Op. Time} \times \text{Op. Count}$$

What would this look like for a **quadratic** algorithm?

$$\text{Time taken} \approx \text{Basic Op. Time} \times (\text{Op. Count})^2$$

How could we find the **basic op. time** for
a **cubic** algorithm?

Order of Growth

Estimated running time for a **linear** algorithm can be expressed as:

$$\text{Time taken} \approx \text{Basic Op. Time} \times \text{Op. Count}$$

What would this look like for a **quadratic** algorithm?

$$\text{Time taken} \approx \text{Basic Op. Time} \times (\text{Op. Count})^2$$

How could we find the **basic op. time** for a **cubic** algorithm?

$$\text{Basic Op. Time} \approx \text{Time taken} \div (\text{Op. Count})^3$$

Example

A large data-crunching algorithm has **quadratic** time complexity, and takes 43 seconds to process 100 pieces of data.

How long will the algorithm take to process 1000 pieces of data?

Example

A large data-crunching algorithm has **quadratic** time complexity, and takes 43 seconds to process 100 pieces of data.

How long will the algorithm take to process 1000 pieces of data?

$$\text{Basic Op. Time} = 43 \div 100^2 = 0.0043\text{s}$$

$$\text{Total Time} = 0.0043 \times 1000^2 = 4300\text{s}$$

Example

A large data-crunching algorithm has **quadratic** time complexity, and takes 43 seconds to process 100 pieces of data.

How long will the algorithm take to process 1000 pieces of data?

Alternatively:
Number of elements has gone up by a factor of 10,
Time taken will go up by a factor of 10^2,
$\text{Total Time} = 43 \times 10^2 = 4300\text{s}$

You Try

1. A searching algorithm with time complexity $O(n)$ searches a list of 1000 items in 5 seconds. How long will it take to search a list of 500 items?
2. A **quadratic** complexity algorithm takes 64 minutes to process 6400 items. How long will it take to process 12800 items?
3. An algorithm takes 500 seconds to process a dataset of 50 items. It then takes 4000 seconds for another dataset of 100 items. State the time complexity of the algorithm.

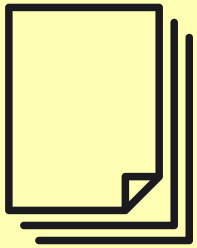
Determining the Time Complexity

To determine the time complexity of an algorithm, we'll need to look for the number of times the **primary operation** is performed, relating to the size of the input.

Look for:

- Loops
- Calculations
- Comparisons

What's the time complexity?



```
static long upperLimit, total;
static void Main(string[] args) {
  Console.WriteLine("Enter upper limit");
  upperLimit = long.Parse(Console.ReadLine());
  total = 0;
  for (int i = 0; i < upperLimit; i++) {
    total += i;
  }
  Console.WriteLine(total);
  Console.ReadKey();
}
```

What's the time complexity?

```
static long upperLimit, total;
static void Main(string[] args) {
  Console.WriteLine("Enter upper limit");
  upperLimit = long.Parse(Console.ReadLine());
  total = 0;
  for (int i = 0; i < upperLimit; i++) {
    total += i;
  }
  Console.WriteLine(total);
  Console.ReadKey();
}
```

Linear Time Complexity, $O(n)$

What's the time complexity?

```
static long upperLimit, total;
static void Main(string[] args) {
  Console.WriteLine("Enter upper limit");
  upperLimit = long.Parse(Console.ReadLine());
  total = 0;
  total = (upperLimit * (upperLimit + 1)) / 2;
  Console.WriteLine(total);
  Console.ReadKey();
}
```

What's the time complexity?

```
static long upperLimit, total;
static void Main(string[] args) {
  Console.WriteLine("Enter upper limit");
  upperLimit = long.Parse(Console.ReadLine());
  total = 0;
  total = (upperLimit * (upperLimit + 1)) / 2;
  Console.WriteLine(total);
  Console.ReadKey();
}
```

Constant Time Complexity $O(1)$

You Try

```
static void findFactors(int n) {
  bool noFactors = true;
  Console.WriteLine($"The factors of {n} are: ");
  for (int i = 2; i < n; i++) {
    if (n % i == 0) {
      Console.WriteLine(i);
      noFactors = false;
    }
  }
  if (noFactors) Console.WriteLine("No factors");
}
```

What if that function was called like so?

What if this is called like so:

```
n = Console.ReadLine()
For i = 2 to 2000
  findfactors(i)
Next
```

More time for your NEA's!

- Testing table should be completed, testing video too.
- Design should have all the headings it needs.
- Some sections of design should be complete.
  - Can you get the High-level or Programming choice sections done?